

Week 4 — Practice Problems (Arrays, Category C) — Solutions

Category C · 5 problems · logic-building with arrays, loops and conditionals only

Name: Gursimran Bajwa | **Registration No:** RA2511003011289

Language: Java | **Problems solved:** 5 / 5

Problem 1 — Two Sum

Problem Statement

A shopkeeper wants to find two items from a list of prices that together add up to exactly a customer's budget. With only a handful of items on the shelf, checking every possible pair is more than fast enough. Exactly one valid pair is guaranteed, and the same element cannot be used twice.

Approach

- Two nested loops walk every pair of positions, with the inner loop starting at $j = i + 1$. That single detail does two jobs: it never pairs an element with itself, and it never checks the same pair twice.
- The moment $\text{nums}[i] + \text{nums}[j]$ equals the target, the indices are returned immediately — no need to scan the rest of the array.
- Time complexity is $O(n^2)$ and space is $O(1)$. For the small inputs described in the scenario this is the right trade-off; a HashMap would give $O(n)$ but is outside the arrays-and-loops constraint.
- An empty array is returned if no pair exists, so the method is still safe on inputs that break the problem's guarantee.

Java Program — TwoSum.java

```
/**
 * L1 - Two Sum (Shopkeeper's Budget Match)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Arrays, loops and conditionals only - no library helpers.
 */
public class TwoSum {

    /* Checks every pair (i, j) with i < j and returns the first pair that hits the target. */
    public static int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[] { i, j }; // exactly one valid pair is guaranteed
                }
            }
        }
        return new int[] {}; // no pair found
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 2, 7, 11, 15 }, { 3, 2, 4 }, { 3, 3 } };
        int[] targets = { 9, 6, 6 };

        for (int t = 0; t < inputs.length; t++) {
            int[] result = twoSum(inputs[t], targets[t]);
        }
    }
}
```

```
        System.out.println("nums = " + format(inputs[t]) + ", target = " + targets[t]);
        System.out.print("  Output: " + format(result));

        if (result.length == 2) {
            System.out.println("    (nums[" + result[0] + "] + nums[" + result[1] + "] = "
                               + inputs[t][result[0]] + " + " + inputs[t][result[1]]
                               + " = " + targets[t] + ")");
        } else {
            System.out.println("    (no valid pair)");
        }
    }
}
```

Output

```
nums = [2, 7, 11, 15], target = 9
  Output: [0, 1]    (nums[0] + nums[1] = 2 + 7 = 9)
nums = [3, 2, 4], target = 6
  Output: [1, 2]    (nums[1] + nums[2] = 2 + 4 = 6)
nums = [3, 3], target = 6
  Output: [0, 1]    (nums[0] + nums[1] = 3 + 3 = 6)
```

Problem 2 — Best Time to Buy and Sell Stock

Problem Statement

A trainee investor has a week of daily stock prices and wants the single best day to buy and the single best later day to sell for the largest possible profit — found in one pass from left to right. If prices only ever fall, the answer is 0.

Approach

- One left-to-right pass tracks two running values: the lowest price seen so far, and the best profit found so far.
- At each day the profit from selling that day is `prices[i] - lowestSoFar`. Because `lowestSoFar` only ever looks backwards, the buy day is always guaranteed to come before the sell day.
- The best profit is updated before the lowest price, so a new minimum on the current day cannot be used as its own sell day.
- `bestProfit` starts at 0, which is exactly why a strictly falling price series correctly returns 0 instead of a negative number.
- $O(n)$ time, $O(1)$ space.

Java Program — `BestTimeToBuyAndSellStock.java`

```
/**
 * L2 - Best Time to Buy and Sell Stock (Trainee Investor)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class BestTimeToBuyAndSellStock {

    /* One left-to-right pass, tracking the lowest price so far and the best profit so far. */
    public static int maxProfit(int[] prices) {
        if (prices.length == 0) return 0;

        int lowestSoFar = prices[0];
        int bestProfit = 0;

        for (int i = 1; i < prices.length; i++) {
            int profitIfSoldToday = prices[i] - lowestSoFar;

            if (profitIfSoldToday > bestProfit) {
                bestProfit = profitIfSoldToday; // better sell day found
            }
            if (prices[i] < lowestSoFar) {
                lowestSoFar = prices[i]; // better buy day found
            }
        }
        return bestProfit; // stays 0 when prices only fall
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 7, 1, 5, 3, 6, 4 }, { 7, 6, 4, 3, 1 }, { 2, 4, 1 } };

        for (int[] prices : inputs) {
            System.out.println("prices = " + format(prices)
                + " -> Max Profit: " + maxProfit(prices));
        }
    }
}
```

Output

```
prices = [7, 1, 5, 3, 6, 4] -> Max Profit: 5
prices = [7, 6, 4, 3, 1] -> Max Profit: 0
prices = [2, 4, 1] -> Max Profit: 2
```



Problem 3 — Contains Duplicate

Problem Statement

Before finalizing an exam seating chart, the office must confirm that no roll number was entered twice — by comparing every entry against every other entry.

Approach

- Nested loops compare each position i against every later position j , so each unordered pair is tested exactly once.
- The method returns true immediately on the first match — an early exit that avoids scanning the remaining pairs once the answer is already known.
- If both loops finish without a match, every value is distinct and false is returned.
- $O(n^2)$ time and $O(1)$ space. A HashSet would bring this to $O(n)$ but is deliberately not used here, since this set is restricted to arrays, loops and conditionals.

Java Program — ContainsDuplicate.java

```
/**
 * L3 - Contains Duplicate (Exam Seating Roll Number Check)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Arrays, loops and conditionals only - no HashSet.
 */
public class ContainsDuplicate {

    /* Compares every pair of different positions and exits early on the first match. */
    public static boolean containsDuplicate(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] == nums[j]) {
                    return true; // early exit - answer is already known
                }
            }
        }
        return false; // every value is distinct
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 1, 2, 3, 1 }, { 1, 2, 3, 4 }, { 1, 1, 1, 3, 3, 4, 3, 2, 4, 2 } };

        for (int[] nums : inputs) {
            System.out.println("nums = " + format(nums)
                + " -> " + containsDuplicate(nums));
        }
    }
}
```

Output

```
nums = [1, 2, 3, 1] -> true
nums = [1, 2, 3, 4] -> false
nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2] -> true
```

Problem 4 — Merge Two Sorted Arrays

Problem Statement

Two class sections each submit exam scores already sorted from lowest to highest. The examination office needs one combined sorted list, without throwing both lists together and re-sorting from scratch.

Approach

- Three indices are kept: i into `arr1`, j into `arr2`, and k as the write position in the result array, which is pre-sized to the combined length.
- A while loop runs as long as both arrays still have elements, copying whichever current element is smaller and advancing only that array's pointer.
- Using $\leq$ rather than $<$ when comparing keeps equal elements in their original relative order, making the merge stable.
- Once one array is exhausted, two tidy-up loops copy whatever remains of the other array — this is also what makes an empty input array work with no special case.
- $O(n + m)$ time, which is the whole point: re-sorting the concatenation would cost $O((n + m) \log(n + m))$.

Java Program — MergeTwoSortedArrays.java

```
/**
 * L4 - Merge Two Sorted Arrays (Combining Two Sections' Exam Scores)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class MergeTwoSortedArrays {

    /* Two-pointer merge: always copy the smaller of the two current elements. */
    public static int[] mergeSortedArrays(int[] arr1, int[] arr2) {
        int[] merged = new int[arr1.length + arr2.length];

        int i = 0;    // pointer into arr1
        int j = 0;    // pointer into arr2
        int k = 0;    // write position in merged

        while (i < arr1.length && j < arr2.length) {
            if (arr1[i] <= arr2[j]) {
                merged[k] = arr1[i];
                i++;
            } else {
                merged[k] = arr2[j];
                j++;
            }
            k++;
        }

        // One array is exhausted - copy whatever is left of the other one.
        while (i < arr1.length) {
            merged[k] = arr1[i];
            i++;
            k++;
        }
        while (j < arr2.length) {
            merged[k] = arr2[j];
            j++;
            k++;
        }

        return merged;
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }
}
```

```
public static void main(String[] args) {  
    int[][][] pairs = {  
        { { 1, 3, 5 }, { 2, 4, 6 } },  
        { { }, { 1, 2, 3 } },  
        { { 1, 2, 9 }, { 3, 4 } }  
    };  
  
    for (int[][] pair : pairs) {  
        System.out.println("arr1 = " + format(pair[0]) + ", arr2 = " + format(pair[1])  
            + " -> " + format(mergeSortedArrays(pair[0], pair[1])));  
    }  
}
```

Output

```
arr1 = [1, 3, 5], arr2 = [2, 4, 6] -> [1, 2, 3, 4, 5, 6]  
arr1 = [], arr2 = [1, 2, 3] -> [1, 2, 3]  
arr1 = [1, 2, 9], arr2 = [3, 4] -> [1, 2, 3, 4, 9]
```

Problem 5 — Rotate Array

Problem Statement

A playlist must be shifted so the last few songs move to the front of the queue, rotating the list to the right by k positions — solved the most direct way: work out where every element lands and build the new order from scratch.

Approach

- k is first reduced with $k = k \% \text{nums.length}$, so rotating by the array's own length (or any multiple) correctly becomes no rotation at all — this is what makes $k = 3$ on a 2-element array behave as a single rotation.
- A new array of the same size is filled by placing each $\text{nums}[i]$ directly at its destination index $(i + k) \% \text{nums.length}$. The modulo is what wraps elements past the end round to the front.
- The rotated values are copied back into nums so the caller's array reflects the change, and the array is also returned for convenience.
- $O(n)$ time and $O(n)$ extra space for the second array.

Java Program — RotateArray.java

```
/**
 * L5 - Rotate Array (Shifting a Playlist Queue)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class RotateArray {

    /* Every element moves to (i + k) % length in a freshly built array. */
    public static int[] rotateArray(int[] nums, int k) {
        if (nums.length == 0) return nums;

        k = k % nums.length; // a full rotation changes nothing
        int[] rotated = new int[nums.length];

        for (int i = 0; i < nums.length; i++) {
            rotated[(i + k) % nums.length] = nums[i];
        }

        // Copy back into nums so the caller's array reflects the rotation too.
        for (int i = 0; i < nums.length; i++) {
            nums[i] = rotated[i];
        }
        return nums;
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 1, 2, 3, 4, 5, 6, 7 }, { 1, 2 }, { 1, 2, 3 } };
        int[] shifts = { 3, 3, 3 };

        for (int t = 0; t < inputs.length; t++) {
            String before = format(inputs[t]);
            String after = format(rotateArray(inputs[t], shifts[t]));
            System.out.println("nums = " + before + ", k = " + shifts[t] + " -> " + after);
        }
    }
}
```

Output

```
nums = [1, 2, 3, 4, 5, 6, 7], k = 3 -> [5, 6, 7, 1, 2, 3, 4]
nums = [1, 2], k = 3 -> [2, 1]
nums = [1, 2, 3], k = 3 -> [1, 2, 3]
```